Lecture 1: Logistic Regression

Andrei A Simion

Columbia University

September 1, 2025

Info

- Me: Andrei Simion
- My Job: Applied Scientist at Goldman Sachs working on Gen AI Applications
- Worked in Tech/Finance for a few years
- PhD IEOR/CS 2015 in Convex (!) Optimization for NLP Mike Collins and Cliff Stein
- Class Policy: Ask stupid questions, please - a stupid question is 100% better than hiding because you are afraid you'll look dumb
  - No one knows everything
  - Please use Courseworks or email me / TAs if you have questions
  - Use Ed! Please!

Schedule of Topics

- Weeks 1 - 6: Logistic Regression, Back Propagation, Optimization, Word2Vec, RNN / GNN / CNN
- Week 7 7/14/25: Midterm
- Weeks 8 - 12: Transformers, BERT, GPT, LORA, DPO/RHLF
- Week 13: Review, catch up, more applications of what we looked at (Information Retrieval, Ads, Scaling Laws)
- Final will be held during the University's scheduled period

Grades

- How will you get a grade?
  - Mostly via HW and Midterm / Final
  - 6 or so assignments, you need to do 6 but it is easy to get full extra credit
    - If you do more than 6, you get extra credit (each HW is worth 10 points) but max score before exams is 70
    - Each assignment is worth the same
  - HW: 60 Midterm: 20 Final: 20
  - You need to take the Midterm and the Final regardless
  - Midterm and Final might be above 20 points; in which case these problems will be extra credit
- HW is due about 2 weeks after it is assigned
- The course objective: know enough NLP / ML from this class so you can impress on a job interview
- If you want: you can write a survey paper and implement a model and/or try to extend some result and submit it: EMNLP / ACL / NAACL
  - I can help with this
  - 5 pages is enough for a "short" paper (can also be for a workshop)

Grades: To get A+, you need to do well on exams and HW

- Score = min(sum(HW + Paper), 70) + Midterm + Final
- Total points example (lots of extra HW but poor on exams): Score = (min(100, 70) + 5 + 5) = 80 = B-
- Total points example (minimal HW but good on exams): Score = (min(58, 70) + 20 + 20) = 98 = A
- If you get a total Score $\sim$ 90 you get A-
- If you get a total Score $\sim$ 100 you get A+ (you did well on exams and did the minimal HW)
- If you get a total Score $\sim$ 93 you get A
- If you get a total Score $\sim$ 82 you get B-
- If you get a Score $\sim$ 78 you get C+
- If you get a Score $\sim$ 62 you get D-
- If you get a Score $<$ 60 you get F
  - Might rescale this is needed to get some bell curve depending on the distribution; but I'll only rescale if it can help people

Office Hours, Ed, Zoom

- My Office Hours (Tentative):
  - After class (in person)
  - Will try for Wednesday before class also - will announce on Ed
- TAs: We have a few and I'm unsure if they will have office hours - need to discuss with them
- I'll try to go about 1.5 hours - 2 hours each week, not 2.5
- Use Ed!!
- You can attend class via Zoom or in person, I don't take attendance

What background / compute do I need?

- Do I need to know lots of Deep Learning / ML for this class?
- Probably not, but you need to know Math + Python (Conda / NumPy) or you can pick up Python
- I'll try to add in things as we go so there a little explanation for different things or a reference you can use to fill in a knowledge gap
- For this class, one goal I have is to go over details of some famous (NLP) papers and not just textbook material
  - I.e. What is Word2Vec?
  - I.e. How does BERT work when you apply it to different tasks? (Let's actually look at the internals) GPT?
- Colab would be great so you have GPU access (especially if you want to do a project), but I'll try to make the work in such a way that you don't need it
- HW is meant for you to understand the basics, not do heavy compute experiments

Outline

- Logistic Regression
- Linear Regression
- Back Propagation, Optimization and Overfitting
- AUC
- PyTorch Notebook Example

Outline

- Logistic Regression
- Linear Regression
- Back Propagation, Optimization and Overfitting
- AUC
- PyTorch Notebook Example

Logistic Regression Data

- Consider Training data $\mathcal{T}$ where $\{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$ are observed and $x^{(i)}$ is some feature vector while $y^{(i)}$ is some response which is always 0/1 valued
- Below, let (x, y) is a generic instance and each observation in $\{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$ is independent of any other
  - Let $x_0 = 1$ always; this is the "intercept"
  - Other x_j for $j > 0$ can be numerical or a part of a one-hot encodings for categorical features
  - For example,

$$x = (1, -2.11, 4.1, 1, 0, 0, 0, 1)$$

or

$$x = (1, 3.14, -0.02, 0, 1, 0, 1, 0)$$

    - We have 2 numerical features and 2 categorical features; one categorical has 3 levels and another has 2 levels
    - Recall one-hot encoding of categorical features: if f has k levels map each instance of f into a $\mathbb{R}^k$ vector equal to 1 at that level and 0 otherwise (we'll see an example later)
- Because y is 1/0, we aim to model y as a Bernoulli Random variable

Logistic Regression Assumptions

- We assume that the probability y depends on x via a linear transformation
- So $y \sim \theta^\intercal x$ but we need a probability for y, since y is $1/0$ and this probability sufficiently explains y
- To get a probability, we use $p(y = 1|x; \theta) = g(\theta^\intercal x)$ where g is some function that maps $\mathbb{R} \to [0, 1]$
- So we have:
  - $\theta^\intercal x$ can be anything, and so the domain of g is $\mathbb{R}$
  - The range of g must be [0, 1] since $g(\theta^\intercal x)$ is a probability
  - This leads to a few variants:
    - Logistic Regression: $g(z) = \sigma(z) = \frac{1}{1+e^{-z}}$
    - Probit Regression: $g(z) = \int_{-\infty}^{z} \frac{e^{-t^2/2}}{\sqrt{2\pi}} dt$

Logistic Regression with σ

- $\sigma(z) = \frac{1}{1+e^{-z}}$ has some nice properties, for example

$$\begin{aligned}\frac{d\sigma(z)}{dz} &= \frac{e^{-z}}{(1+e^{-z})^2} \\ &= (\frac{e^{-z}}{1+e^{-z}})(\frac{1}{1+e^{-z}}) \\ &= \sigma(z)(1-\sigma(z))\end{aligned}$$

- Note also that σ is monotonic in z
- This implies, for example that we have $p(y=1|x;\theta) = \sigma(\theta^\intercal x)$ increasing in each θ_j if all others are fixed
  - Note that this model is easy to interpret: if $\theta_j > 0$ this means that as x_j gets larger $p(y=1|x;\theta)$ gets larger

One-Hot representation: review this more if you need it

- Features f that are categorical can be expressed as one-hot vectors: if f takes on 4 values f is a vector of 1/0 values and exactly one of these values is 1
- Similarly, the response for logistic regression can be written as a one-hot vector: 0 can map to (1, 0) and 1 can map to (0, 1)
- One-hot representations are important since they allow us to represent a categorical feature or response as a succinct vector
- But they have a problem
  - Assume we have a feature f that has 4 categories and categories are very much related (say "Math" = 0, "Physics" = 1, "Painting" = 2, "Writing" = 3)
  - If we have two instances $f^{(1)}$ = "Math" and $f^{(2)}$ = "Physics" which are have one-hot representations (1, 0, 0, 0) and (0, 1, 0, 0) then $(f^{(1)})^\intercal f^{(2)} = 0$
  - But, this might not make sense: ideally the inner product captures some "similarity" between the two vectors since Math and Physics are related

Outline

- Logistic Regression
- Linear Regression
- Back Propagation, Optimization and Overfitting
- AUC
- PyTorch Notebook Example

Linear Regression Assumptions

- Here, we want to predict y from x but the y is continuous and real valued
- In this case, we assume that $y = \theta^\intercal x + e$ where e is a normal mean 0 variance σ^2 random variable
  - The parameters are (θ, σ^2)
  - The above implies y is a normal random variable with mean $\theta^\intercal x$ and variance σ^2
  - This means $p(y|x;\theta,\sigma^2) = \frac{e^{\frac{-(y-\theta^\intercal x)^2}{2\sigma^2}}}{\sqrt{2\pi\sigma^2}}$ so that

$$\log p(y|x;\theta,\sigma^2) = -(y-\theta^\intercal x)^2/2\sigma^2 - \log\sqrt{2\pi\sigma^2}$$

- We again assume that given training data $\{(x^{(i)}, y^{(i)})\}_{i=1}^N$ each observation pair is independent of any other

From all this, you can derive the likelihood of the data

$$\mathcal{L}(\theta,\sigma^2) = p(y^{(1)},\ldots,y^{(N)}|x^{(1)},\ldots,x^{(N)};\theta,\sigma^2) = \prod_{i=1}^N p(y^{(i)}|x^{(i)};\theta,\sigma^2)$$

and the goal is $\max_{\theta,\sigma^2} \mathcal{L}(\theta,\sigma^2)$

Linear Regression: Log-Likelihood

- The above simplifies a bit to give

$$\mathcal{L}(\theta, \sigma^2) = \prod_{i=1}^{N} \frac{e^{-(y^{(i)} - \theta^\intercal x^{(i)})^2/2\sigma^2}}{\sqrt{2\pi\sigma^2}}$$

  and the goal is $\max_{\theta,\sigma^2} \mathcal{L}(\theta, \sigma^2)$

- From this, you can derive the log-likelihood of the data

$$\ell(\theta, \sigma^2) = \log \mathcal{L}(\theta, \sigma^2) = [\sum_{i=1}^{N} -(y^{(i)} - \theta^\intercal x^{(i)})^2/2\sigma^2] - \frac{N}{2} \log 2\pi\sigma^2$$

  and the goal is $\max_{\theta,\sigma^2} \ell(\theta, \sigma^2)$

- Since log is increasing, notice that

$$\max_{\theta,\sigma^2} \mathcal{L}(\theta, \sigma^2) \sim \max_{\theta,\sigma^2} \ell(\theta, \sigma^2)$$

  and the max is attained at the same point

Linear Regression: Loss

We have

$$\ell(\theta, \sigma^2) = \log \mathcal{L}(\theta, \sigma^2) = [\sum_{i=1}^{N} -(y^{(i)} - \theta^\intercal x^{(i)})^2/2\sigma^2] - \frac{N}{2} \log 2\pi\sigma^2$$

and the goal is $\max_{\theta,\sigma^2} \ell(\theta, \sigma^2)$

- If you focus on just the the part dependent on θ, it is clear that we want to minimize

$$\mathcal{J}(\theta) = \sum_{i=1}^{N} (y^{(i)} - \theta^\intercal x^{(i)})^2$$

  so that the goal is $\min_\theta \mathcal{J}(\theta)$
- Typically, since $\mathcal{J}(\theta)$ is minimized, we call this a loss and $\min_\theta \mathcal{J}(\theta)$ is equivalent to $\max_\theta \ell(\theta, \sigma^2)$
- The optimization for σ^2 is easy, no need to make a loss vs log-likelihood connection

Outline

- Logistic Regression
- Linear Regression
- Back Propagation, Optimization and Overfitting
- AUC
- PyTorch Notebook Example

Optimization for Logistic Regression

- Back to Logistic Regression
- How do we optimize for θ?
- Typically, we have data $\{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$ and we write down the likelihood function, then we optimize
- Assume we have

$$p(y = 1|x; \theta) = \sigma(\theta^{\intercal} x)$$
$$p(y = 0|x; \theta) = 1 - \sigma(\theta^{\intercal} x)$$

- Note that this can be written very easily as

$$p(y|x; \theta) = \sigma(\theta^{\intercal} x)^{y} (1 - \sigma(\theta^{\intercal} x))^{1-y}$$

Optimization for Logistic Regression

- One assumption of Logistic Regression is that for each i training samples are independent
- This means that if we write the likelihood of the data we have

$$p(y^{(1)}, \ldots, y^{(N)} | x^{(1)}, \ldots, x^{(N)}; \theta) = \prod_{i=1}^{N} p(y^{(i)} | x^{(i)}; \theta)$$

- This simplifies more to

$$\mathcal{L}(\theta) = \prod_{i=1}^{N} p(y^{(i)} | x^{(i)}; \theta) = \prod_{i=1}^{N} \sigma(\theta^{\mathsf{T}} x^{(i)})^{y^{(i)}} (1 - \sigma(\theta^{\mathsf{T}} x^{(i)}))^{1-y^{(i)}}$$

- As with Linear Regression, we want to find

$$\max_{\theta} \mathcal{L}(\theta)$$

How do we optimize $\ell(\theta)$

- As before, we take the logarithm and set the log-likelihood to $\ell(\theta) = \log \mathcal{L}(\theta)$ and we have

$$\max_{\theta} \mathcal{L}(\theta) \sim \max_{\theta} \ell(\theta)$$

- We get

$$\ell(\theta) = \sum_{i=1}^{N} [y^{(i)} \log \sigma(\theta^{\mathsf{T}} x^{(i)}) + (1 - y^{(i)}) \log 1 - \sigma(\theta^{\mathsf{T}} x^{(i)})]$$

- Some Questions:
  - How do you optimize this function?
  - What do the gradients look like?
  - What's the general idea and can we eventually build on this idea for more complicated (Neural) models?

Binary Cross-Entropy Step by Step

- For y $1/0$ and p some probability which tries to match $y \sim p$
- Then, the Cross-Entropy between y and p is

$$-y \log p - (1 - y) \log(1 - p)$$

  - If p is close to y, this Cross-Entropy is 0 and the number above represents how far p is from the truth y
- Consider one term $-y \log a - (1 - y) \log(1 - a)$ for some (x, y) so $p = a = \sigma(\theta^\intercal x)$
  - Suppose we have some x and have a true label $y = 1$ without loss of generality
  - Then, the above becomes $-\log a$
  - This is the term controlled by the model: the model, based on its parameters θ, sets a since $a = \sigma(z) = \sigma(\theta^\intercal x)$
  - A good model is such that in the above case a is close to 1, since we want to minimize and $0 \leq a \leq 1$
  - Similarly, you can argue that if $y = 0$ then a good model has $a \sim 0$ since we need to minimize $-\log 1 - a$

Binary Cross-Entropy For Entire Dataset

- Take $z^{(i)} = \theta^\intercal x^{(i)}$
- Then the Cross-Entropy $\mathcal{J}(\theta)$ is the negative of the log-likelihood $\ell(\theta)$ across the data and

$$\mathcal{J}(\theta) = -\ell(\theta) = -\sum_{i=1}^{N} [y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log 1 - a^{(i)}]$$

- Essentially, for each i we are modeling $p(y^{(i)} = 1 | x^{(i)}; \theta)$ and we want to push θ as needed so $a^{(i)} = \sigma(\theta^\intercal x^{(i)})$ is close to $y^{(i)}$ on average across all i
- So, the goal of Logistic Regression is to find a θ that is good on average across all points $\{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$
- When $y^{(i)}$ is 1 we want $\sigma(\theta^\intercal x^{(i)})$ to also be 1 and similarly for $y^{(i)} = 0$

Gradient Descent / Ascent and Optimizing $\mathcal{J}(\theta)$

- To do an optimization of the form $\min_\theta \mathcal{J}(\theta)$ we can use Full (Batch) Gradient Descent
- The algorithm goes as follows:
  - Pick a learning rate $\rho > 0$
  - Pick an initial value θ_0
  - For $t = 1, \ldots, T$
    - Set $\theta_t = \theta_{t-1} - \rho \nabla \mathcal{J}(\theta_{t-1})$
  - Return θ_T
- There are other variants of this, but at a high level this is the idea
- Recall we have

$$-\max_\theta \ell(\theta) \sim \min_\theta \mathcal{J}(\theta)$$

- For the log-likelihood maximization the update rule looks like $\theta_t = \theta_{t-1} + \rho \nabla \ell(\theta_{t-1})$ where we want to solve $\max_\theta \ell(\theta)$

Mini Batch Gradient Descent

- To do an optimization of the form $\min_\theta \mathcal{J}(\theta)$ we usually use Mini Batch Gradient Descent
- This is needed to speed things up - we update θ much more often!
- Big idea: break up the Training data $\mathcal{T}$ into batched $\mathcal{T}_1, \ldots, \mathcal{T}_K$ which are disjoint and cover $\mathcal{T}$
- At a high level, the algorithm goes as follows:
- Pick a learning rate $\rho > 0$
- Pick an initial value $\theta = \theta_0$
- For $t = 1, \ldots, T$
  - For $k = 1, \ldots, K$
    - Get the loss $\mathcal{J}^{\mathcal{T}_k}(\theta)$ over just this batch $\mathcal{T}_k$
    - Denote by $\nabla \mathcal{J}^{\mathcal{T}_k}(\theta)$ the gradient of this loss just over this batch
    - Set $\theta = \theta - \rho \nabla \mathcal{J}^{\mathcal{T}_k}(\theta)$
- Return θ

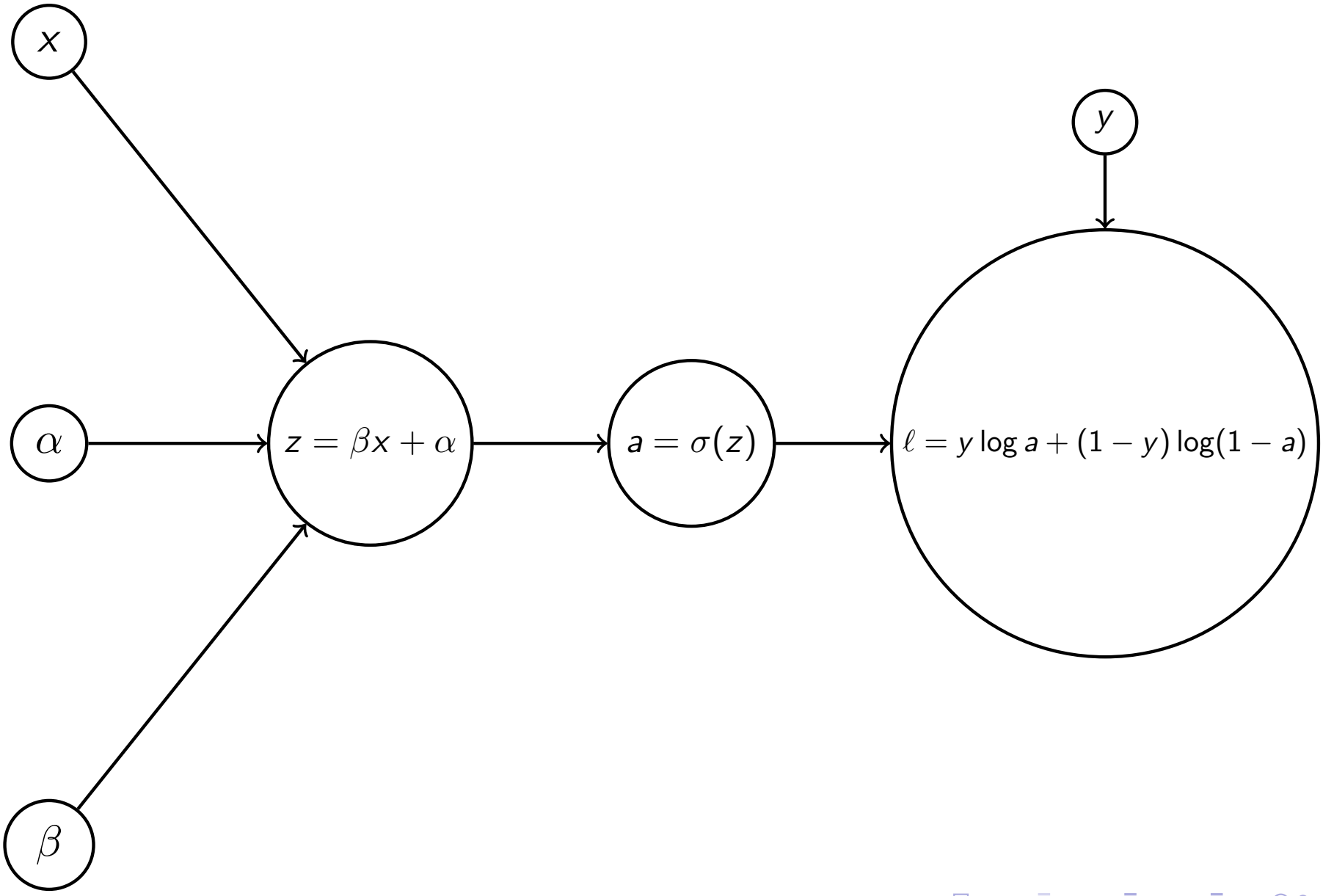
Gradients for Logistic Regression

- The core idea lies in finding gradients: $\mathcal{J}^{\mathcal{T}_k}(\theta)$ and similar terms
- How do we get the gradients with data? Do we need them analytically always?
- No! In PyTorch / Tensorflow there are numerical methods that give us the gradients automatically; we'll see some examples as we go
- But it is good to know what the gradients are so we understand what is going on
- What do the gradients look like for Logistic Regression?
- Consider just one training example (x, y)

Logistic Regression Gradients: Let's compute them in a general way!

- Let's consider the maximization $\max_\theta \ell(\theta)$
- What is $\frac{\partial \ell}{\partial \theta_j(\theta)}$?
- Let's try to do this with something called a computation graph, since this is what we'll need for Neural Networks
- Assume x has dimension 1 for the sake of argument, so that $\theta^\intercal x = \beta x + \alpha$ and we have just two parameters $\beta \in \mathbb{R}, \alpha \in \mathbb{R}$
- How do we write out the dependencies between things as a graph?
  - We have 3 "leaves", x, α, β
  - We have $z = \beta x + \alpha$
  - We have $a = \sigma(z)$
  - We have $\ell(\alpha, \beta) = y \log a + (1 - y) \log(1 - a)$ and we want $\frac{\partial \ell}{\partial \alpha}$ and $\frac{\partial \ell}{\partial \beta}$ so we can do gradient optimization

Logistic Regression Computational Graph: What is the flow of data?



Logistic Regression Computational Graph

- Note: this is a Directed Acyclic Graph (DAG)
  - The parameters and data are the leaves: they have no "in" arcs
  - There are no cycles
  - Generally, this will be how graphs look like and these graphs help us reason about what the chain rule gives so we can get gradients
- How do we find $\frac{\partial \ell}{\partial \alpha}$?
  - The chain rule can be written as starting from ℓ and considering all paths to α
  - There is just 1 path in this case
  - $\alpha \to z \to a \to \ell$
  - This mean that
$$\frac{\partial \ell}{\partial \alpha} = (\frac{\partial \ell}{\partial a})(\frac{\partial a}{\partial z})(\frac{\partial z}{\partial \alpha})$$
  - Note that you must remember the chain rule ... In the above
    - On the left, we have gradients
    - On the right, we have local gradients

Logistic Regression Computational Graph

- We have
$$\frac{\partial \ell}{\partial \alpha} = (\frac{\partial \ell}{\partial a})(\frac{\partial a}{\partial z})(\frac{\partial z}{\partial \alpha})$$
- We have $\ell = y \log a + (1-y)\log(1-a)$
  - This means $\frac{\partial \ell}{\partial a} = \frac{y}{a} - \frac{1-y}{1-a} = \frac{y-a}{a(1-a)}$
- We have $a = \sigma(z)$
  - This means $\frac{\partial a}{\partial z} = \sigma(z)(1-\sigma(z)) = a(1-a)$
- We have $z = \beta x + \alpha$
  - This means $\frac{\partial z}{\partial \alpha} = 1$
- This means
$$\frac{\partial \ell}{\partial \alpha} = (\frac{y-a}{a(1-a)})(a(1-a)) = y - a$$
- Similarly, you can show that
$$\frac{\partial \ell}{\partial \beta} = (\frac{y-a}{a(1-a)})(a(1-a))x = (y-a)x$$

Logistic Regression Computational Graph

- There is an recipe in all this
- To do Gradient Descent / Ascent you need gradients and gradients can be gotten by considering a graph like before:
  - Fix your current parameter $\theta_{old} = (\alpha_{old}, \beta_{old})$
  - Draw DAG: write down the graph above to understand the relationships between steps
  - Forward Pass: compute z, a, ℓ as above numerically by plugging in $x, \alpha_{old}, \beta_{old}$ and going forward
  - Backward Pass: use the graph above (and the values) to compute $\frac{\partial \ell}{\partial \alpha}(\alpha_{old}, \beta_{old})$ and $\frac{\partial \ell}{\partial \beta}(\alpha_{old}, \beta_{old})$
  - Update: Use the above to update parameters; we have

$$\alpha_{new} = \alpha_{old} + \rho \frac{\partial \ell}{\partial \alpha}(\alpha_{old})$$

and

$$\beta_{new} = \beta_{old} + \rho \frac{\partial \ell}{\partial \beta}(\beta_{old})$$

  - Repeat ...
  - For example, we could have done batch gradient ascent with batches of size 1; the above is exactly what we would do!

Another Example: Overfitting and the DAG for this case

- When we fit a model to the training data, we usually don't just care about Training performance
- We also care about performance on another separate Test set
- Typically, to control this, we add a regularization term $\mathcal{R}(\theta) \geq 0$ which is positive and which controls model complexity
  - The 2-norm is $\mathcal{R}(\theta) = \sum_{j=1}^{d} \theta_j^2$ and typically this makes θ_j small but $\neq 0$
  - The 1-norm is $\mathcal{R}(\theta) = \sum_{j=1}^{d} |\theta_j|$ and typically this makes θ sparse so that very few θ_j are nonzero
- Given the regularization, we fix $\lambda > 0$ and then focus on $\max_\theta \ell(\theta) - \lambda \mathcal{R}(\theta)$
  - Note that, the larger the λ the more we focus on $\mathcal{R}(\theta)$
  - If $\lambda \to \infty$ then we get $\theta = 0$ since $\mathcal{R}(\theta)$ dominates
  - The right λ usually needs to be tuned in some way; it is a hyperparameter

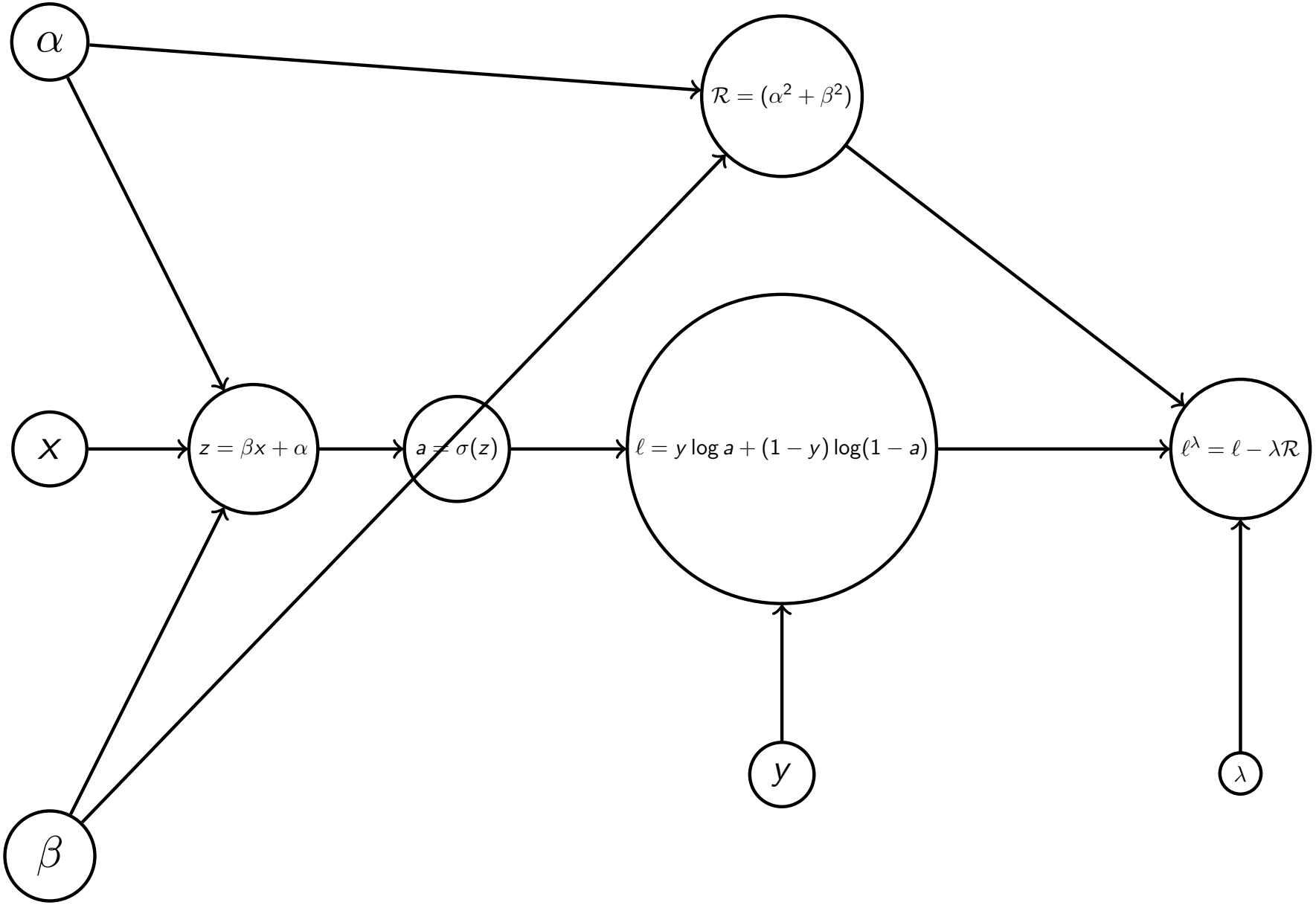
Computational Graph For Logistic Regression with Regularization

- Assume we pick $\mathcal{R}(\theta) = \sum_{j=1}^{d} \theta_j^2$ so that the optimization we consider is

$$\max_{\theta} \ell(\theta) - \lambda(\sum_{j=1}^{d} \theta_j^2) = \max_{\theta} \ell^{\lambda}(\theta)$$

- Assume $\theta = (\alpha, \beta) \in \mathbb{R}^2$ as before so that $\mathcal{R}(\theta) = \alpha^2 + \beta^2$
- What does the computational graph look like?
- What are the gradients?
- We'll do this very carefully, so that the last node is ℓ^{λ}

Regularized Logistic Regression: Computational Graph



Computing the Gradients again

- What is $\frac{\partial \ell^\lambda}{\partial \alpha}$?
- In this case, we have two paths
  - $\alpha \rightarrow z \rightarrow a \rightarrow \ell \rightarrow \ell^\lambda$
  - $\alpha \rightarrow \mathcal{R} \rightarrow \ell^\lambda$
- We have

$$\frac{\partial \ell^\lambda}{\partial \alpha} = (\frac{\partial \ell^\lambda}{\partial \ell})(\frac{\partial \ell}{\partial a})(\frac{\partial a}{\partial z})(\frac{\partial z}{\partial \alpha}) + (\frac{\partial \ell^\lambda}{\partial \mathcal{R}})(\frac{\partial \mathcal{R}}{\partial \alpha})$$

Computing the Gradients again

- We have
  $\ell^\lambda = \ell - \lambda\mathcal{R} = y \log a + (1-y)\log(1-a) - \lambda(\alpha^2 + \beta^2)$
  - This means $\frac{\partial \ell^\lambda}{\partial a} = (\frac{\partial \ell^\lambda}{\partial \ell})(\frac{\partial \ell}{\partial a}) = (1)(\frac{y}{a} - \frac{1-y}{1-a}) = \frac{y-a}{a(1-a)}$
  - On the other hand we have the local $\frac{\partial \ell^\lambda}{\partial \mathcal{R}} = -\lambda$ and $\frac{\partial \mathcal{R}}{\partial \alpha} = 2\alpha$
- We have $a = \sigma(z)$
  - This means $\frac{\partial a}{\partial z} = \sigma(z)(1-\sigma(z)) = a(1-a)$
- We have $z = \beta x + \alpha$
  - This means $\frac{\partial z}{\partial \alpha} = 1$
- This means

$$\frac{\partial \ell^\lambda}{\partial \alpha} = (\frac{y-a}{a(1-a)})(a(1-a)) - 2\lambda\alpha = y - a - 2\lambda\alpha$$

- Similarly, you can derive $\frac{\partial \ell^\lambda}{\partial \beta}$

Computing the Gradients again

- This Computational Graph idea is very useful
- At a high level, this is just the chain rule but the graph helps us reason about the terms we need
- We'll see many more examples in the class but it's crucial that you understand this example

Outline

- Logistic Regression
- Linear Regression
- Back Propagation, Optimization and Overfitting
- AUC
- PyTorch Notebook Example

AUC: A Review

- What makes a model good at classification for 1/0 response?
- Logistic Regression gives us two types of probabilities, $p(y = 1)$ and $p(y = 0)$
- Imagine you have a imbalanced data, where 95% of the data is $y = 0$ and $y = 1$ is only 5% of the data
- Accuracy is not good here, since trivial classifiers give good results
  - A random classifier (predicting $y = 1$ with 5%) would give an accuracy of approximately $\sim 90\%$
  - A deterministic classifier that always predicts 0 would give an accuracy of 95% too
- Other metrics, like Recall and Precision came about to address this; an intuitive one you should know is AUC

AUC

- There are a few ways to understand AUC (i.e. with ROC graphics) but this one you might not know
- What should a good model do?
  - Generally, we would hope it gives a higher score $p(y = 1)$ on average to response cases with $y = 1$ (positives) than with $y = 0$ (negatives)
- This is the idea of the AUC; let $\mathcal{P}$ and $\mathcal{N}$ be the positive and negative samples we have in a dataset and g be our model, which returns $p(y = 1)$
- A good model, intuitively, gives class 1 scores close to 1 for $y \in \mathcal{P}$ and close to zero for $y \in \mathcal{N}$
  - For a Logistic Regression, this is $g(x) = \sigma(\theta^\intercal x)$ and notice that this is $p(y = 1)$
  - Let 1_A be the indicator of A: 1 if A is true and 0 otherwise
- We then have

$$\text{AUC} = \frac{\sum_{i \in \mathcal{P}} \frac{\sum_{j \in \mathcal{N}} 1_{\{g(x^{(i)}) > g(x^{(j)})\}}}{|\mathcal{N}|}}{|\mathcal{P}|}$$

AUC Intuition

$$\text{AUC} = \frac{\sum_{i \in \mathcal{P}} \frac{\sum_{j \in \mathcal{N}} 1_{\{g(x^{(i)}) > g(x^{(j)})\}}}{|\mathcal{N}|}}{|\mathcal{P}|}$$

- Put another way:
  - A good model should score points in $\mathcal{P}$ higher than it does points in $\mathcal{N}$
  - The AUC as expressed above is an average, where we consider the average number of times a particular positive point is higher than all negative ones
  - Then, we average over all positive points
- What is a bad AUC?
- If you consider the above, you can reason that a 50% value is bad, as it would be the AUC of a random classifier

Why don't we just optimize AUC?

- If we care about AUC, why don't we just optimize it directly?
- $$\text{AUC} = \frac{\sum_{i \in \mathcal{P}} \frac{\sum_{j \in \mathcal{N}} 1_{\{g(x^{(i)}) > g(x^{(j)})\}}}{|\mathcal{N}|}}{|\mathcal{P}|}$$
- The above does not have a gradient that's easy to compute
- Loosely, a higher Log-Likelihood leads to a higher AUC
- Crucially, the Log-Likelihood is much easier to optimize

AUC in PyTorch and leading into Examples

```python
# Computing AUCROC
from torch import tensor
from torchmetrics import AUROC
preds = tensor([0.13, 0.26, 0.08, 0.19, 0.34])
target = tensor([0, 0, 1, 1, 1])
auroc = AUROC(task="binary")
auroc(preds, target)
```

```
tensor(0.5000)
```

- For now, think of a tensor as a type of NumPy array
- Here, $|\mathcal{P}| = 3$ and $|\mathcal{N}| = 2$
- Try to compute this manually to make sure you understand it

What if we reverse $y = 1$ and $y = 0$

- Notice that AUC is computed with respect to the class $y = 1$

$$\text{AUC} = \frac{\sum_{i \in \mathcal{P}} \frac{\sum_{j \in \mathcal{N}} 1_{\{g(x^{(i)}) > g(x^{(j)})\}}}{|\mathcal{N}|}}{|\mathcal{P}|}$$

- For class $y = 0$, you can do the same but the equation becomes as below since $1 - g(x)$ is the probability that $y = 0$

$$\text{AUC} = \frac{\sum_{j \in \mathcal{N}} \frac{\sum_{i \in \mathcal{P}} 1_{\{1-g(x^{(j)}) > 1-g(x^{(i)})\}}}{|\mathcal{P}|}}{|\mathcal{N}|}$$

- This is since for a true class $y = 0$ we expect the 0 score for these $1 - g(x)$ to be larger than the zero scores for a true class $y = 1$

But this is the same thing

- The above formula is

$$\text{AUC} = \frac{\sum_{j \in \mathcal{N}} \frac{\sum_{i \in \mathcal{P}} 1_{\{1-g(x^{(j)})>1-g(x^{(i)})\}}}{|\mathcal{P}|}}{|\mathcal{N}|}$$

- This is the same as (rearranging inequalities)

$$\text{AUC} = \frac{\sum_{j \in \mathcal{N}} \frac{\sum_{i \in \mathcal{P}} 1_{\{g(x^{(i)})>g(x^{(j)})\}}}{|\mathcal{P}|}}{|\mathcal{N}|}$$

- Which is the same as (swamp sums)

$$\text{AUC} = \frac{\sum_{i \in \mathcal{P}} \frac{\sum_{j \in \mathcal{N}} 1_{\{g(x^{(i)})>g(x^{(j)})\}}}{|\mathcal{N}|}}{|\mathcal{P}|}$$

- These are the same thing!

What about Multiclass AUC?

- Generally, for a model predicting K classes, we get a vector of probabilities $p^{(i)} = (p_1^{(i)}, p_2^{(i)}, ..., p_K^{(i)})$ for each $x^{(i)}$
- Here, K is the number of classes
- How do we get an AUC in this case?
  - Fix a class k and for each point i take just $p_k^{(i)}$
  - Label each $y^{(i)}$ as $1/0$ where 1 means the true class is k and 0 is anything else
  - Compute a AUC_k which is like the AUC in 2 dimensions for probability $p_k^{(i)}$ and the vector $y^{(i)}$
  - Do this for all k
  - Average over all the $k = 1, \ldots, K$
- This is your Multiclass AUC,

$$\text{AUC} = \frac{\sum_{k=1}^{K} \text{AUC}_k}{K}$$

How do you use AUC?

- Say you have 3 positive samples and 987 negative samples
- You see 97 % AUC; how do you interpret this?
- On average, across the 3 samples, this means that the score of one of the positives is larger than 97 % of the score given to true negatives
- So, this means that on average, for each positive samples, we expect to have 0.97 * 987 = 957 negatives we score higher than
- So, this means that there are roughly 30 negatives for each positive that have a higher probability score given by the model; the model is breaking down for these

Outline

- Logistic Regression
- Linear Regression
- Back Propagation, Optimization and Overfitting
- AUC
- PyTorch Notebook Example

Example in PyTorch

Let's look at a notebook quickly!

References

- CS 229 Lecture Notes by Andrew Ng
- PyTorch + NLP book